

Relatório T1

Projeto e Otimização de Algoritmos

Integrantes: Bernardo Fensterseifer, Bernardo Klein Heitz, Joao Pedro Aiolfi e Rafael Toneto

1. Introdução

Neste trabalho estudamos, implementamos e analisamos o problema da Maior Subsequência Comum, conhecido pela sigla LCS, do inglês *Longest Common Subsequence*. O objetivo do problema é, dadas duas strings, determinar o tamanho da maior sequência de caracteres que aparece nas duas, mantendo a ordem relativa dos caracteres, mas sem exigir que eles estejam em posições consecutivas.

O problema de LCS é um exemplo clássico de aplicação de programação dinâmica pois apresenta subproblemas sobrepostos e estrutura ótima. A mesma pergunta sobre prefixos menores das strings aparece muitas vezes durante uma solução recursiva direta. Programação dinâmica evita repetição desses cálculos ao armazenar resultados intermediários.

Neste projeto, foram implementadas três abordagens:

- algoritmo recursivo puro, usado como versão de força bruta;
- algoritmo com programação dinâmica bottom-up;
- algoritmo com memoização top-down, usado como comparação adicional.

Também foram incluídos testes automatizados, contagem de operações, medição de tempo de execução, reconstrução de uma subsequência comum máxima e geração de resultados em CSV.

2. Descrição do problema

Uma subsequência de uma string é formada pela remoção de zero ou mais caracteres, preservando a ordem dos caracteres restantes. Diferentemente de uma substring, uma subsequência não precisa ocupar posições consecutivas.

Por exemplo, para a string ABCBDAB, a sequência BCBA é uma subsequência, pois seus caracteres aparecem nessa ordem na string original. Já a sequência BAC não é uma subsequência dessa string se a ordem exigida não puder ser preservada.

O problema da LCS recebe duas strings, aqui chamadas de s_1 e s_2 , e procura o maior tamanho possível de uma subsequência que exista nas duas. O comprimento da LCS é único, mas a subsequência em si pode não ser única. Em alguns casos, existem várias respostas diferentes com o mesmo tamanho máximo.

3. Exemplos de casos do problema

s_1	s_2	Comprimento da LCS	Uma LCS possível
ABCBDA	BDCABA	4	BCBA
AGGTAB	GXTXAYB	4	GTAB
AAAA	AAAA	4	AAAA
ABCD	EFGH	0	string vazia
string vazia	ABC	0	string vazia
abcdefghij	acegikmoq	5	acegi

Esses exemplos mostram situações diferentes: strings iguais, strings sem caracteres em comum, entrada vazia e casos em que apenas parte dos caracteres pode ser aproveitada.

4. Algoritmo recursivo de força bruta

A solução recursiva considera prefixos das duas strings. Seja $LCS(i, j)$ o comprimento da maior subsequência comum entre os i primeiros caracteres de $s1$ e os j primeiros caracteres de $s2$.

A recorrência usada é:

- se $i = 0$ ou $j = 0$, então uma das strings consideradas está vazia, logo $LCS(i, j) = 0$;
- se $s1[i - 1] = s2[j - 1]$, os últimos caracteres dos prefixos são iguais, então eles podem fazer parte da resposta e $LCS(i, j) = 1 + LCS(i - 1, j - 1)$;
- caso contrário, os últimos caracteres são diferentes, então a resposta é o melhor entre ignorar o último caractere de $s1$ ou ignorar o último caractere de $s2$: $LCS(i, j) = \max(LCS(i - 1, j), LCS(i, j - 1))$.

Essa abordagem é simples e mostra diretamente a definição matemática do problema. Porém, ela recalcula muitos subproblemas. Por exemplo, ao comparar ABCBDAB com BDCABA, chamadas como $LCS(5, 4)$ podem ser alcançadas por mais de um caminho da árvore de recursão. Esse comportamento causa crescimento exponencial no pior caso.

No código, a versão recursiva está no método `lcsRecursive`, da classe `LcsAlgorithms`. A contagem de operações dessa abordagem considera uma operação a cada entrada na função recursiva, incluindo chamadas que chegam ao caso base.

Complexidade: no pior caso, o tempo é exponencial em relação aos tamanhos das strings. O espaço adicional é proporcional à profundidade da pilha de recursão, isto é, $O(m + n)$.

5. Algoritmo com programação dinâmica

A solução com programação dinâmica bottom-up usa uma matriz `dp` de tamanho $(m + 1) \times (n + 1)$, em que m é o tamanho de $s1$ e n é o tamanho de $s2$.

Cada célula `dp[i][j]` guarda o comprimento da LCS entre os prefixos:

- $s1[0..i-1]$;
- $s2[0..j-1]$.

A linha 0 e a coluna 0 representam comparações com uma string vazia, portanto ficam com valor 0. Depois disso, a matriz é preenchida linha por linha. Para cada par de posições:

- se os caracteres atuais são iguais, a resposta vem da diagonal anterior mais 1;
- se os caracteres são diferentes, a resposta vem do maior valor entre a célula de cima e a célula da esquerda.

Essa estratégia elimina os recálculos da recursão pura, porque cada subproblema `dp[i][j]` é resolvido uma única vez. Ao final, a resposta fica em `dp[m][n]`.

No código, essa abordagem está no método `lcsDp`, também da classe `LcsAlgorithms`. A contagem de operações considera uma operação para cada célula interna processada, ou seja, para cada par (i, j) com $i \geq 1$ e $j \geq 1$. Assim, o número de operações da versão DP tende a ser $m * n$.

Complexidade: o tempo é $O(m * n)$, pois todas as células internas da matriz são preenchidas. O espaço também é $O(m * n)$, devido ao armazenamento da tabela.

6. Memoização como comparação adicional

A implementação também inclui uma versão com memoização. Ela usa a mesma recorrência do algoritmo recursivo, mas armazena em uma matriz `memo` o resultado de cada estado já calculado.

Essa abordagem é top-down: a execução ainda começa no problema completo, mas evita recomputar estados repetidos. Quando uma chamada recursiva tenta resolver um estado que já foi calculado, o valor é retornado diretamente da tabela.

No código, essa versão está no método `lcsMemo`. A contagem de operações considera uma entrada no helper recursivo, inclusive quando o valor é encontrado no cache. Por isso, a contagem de operações da memoização não

deve ser comparada de forma absoluta com a contagem da programação dinâmica bottom-up, pois as definições de operação são diferentes.

Complexidade: no pior caso, o tempo é $O(m * n)$ e o espaço é $O(m * n)$, além da pilha de recursão.

7. Descrição da implementação

O projeto foi desenvolvido em Java, usando Maven para compilação, execução e testes. A organização principal do código é:

Arquivo	Responsabilidade
src/main/java/com/poa/lcs/LcsAlgorithms.java	Implementa LCS recursivo, LCS com DP, LCS com memoização e reconstrução de uma subsequência máxima.
src/main/java/com/poa/lcs/OperationCounter.java	Fornecer um contador simples para registrar operações durante as execuções.
src/main/java/com/poa/lcs/BenchmarkRunner.java	Define os casos de benchmark, mede tempos, contabiliza operações e exporta resultados em CSV.
src/main/java/com/poa/lcs/Main.java	Controla a execução pela linha de comando, imprime os resultados e chama o benchmark.
src/test/java/com/poa/lcs/LcsAlgorithmsTest.java	Testa a correção dos algoritmos em casos conhecidos.
src/test/java/com/poa/lcs/LcsPerformanceTest.java	Testa concordância entre DP e memoização em maior escala e compara os três métodos em casos pequenos.

A classe Main aceita dois modos de uso. Sem argumentos, executa uma lista de casos padrão. Com dois argumentos, trata esses argumentos como as duas strings de entrada. Quando a flag --csv é usada, o programa também grava o arquivo results/benchmark.csv.

Para evitar que o algoritmo recursivo puro torne o benchmark inviável em entradas maiores, a classe BenchmarkRunner define o limite RECURSIVE_MAX_SUM_LENGTH = 25. Quando a soma dos tamanhos das duas strings passa desse limite, a versão recursiva é marcada como n/a.

A função reconstructLcs monta uma subsequência comum máxima a partir da matriz de programação dinâmica. Ela começa na célula final da tabela e caminha de volta até a origem, adicionando caracteres quando encontra uma correspondência entre as duas strings. Como o caminho é percorrido do fim para o início, a string final é invertida antes de ser retornada.

8. Exemplos de execução

O projeto contém prints de execução na pasta images.

Execução com os casos padrão:

```

PS C:\Users\joxto\Downloads\T1-ProjetoOtimizacaoAlgoritmos\scripts> cd ..
PS C:\Users\joxto\Downloads\T1-ProjetoOtimizacaoAlgoritmos> .\mvnw.cmd -q compile exec:java
=====
lcs ? recursivo | dp (bottom-up) | memoização
=====
limiar recursivo: |s1|+|s2| <= 25 (acima: n/a)

caso "classico" ? strings: "ABCBDB" e "BDCABA"
comprimento lcs: 4 | uma subsequência: "BCBA"
---
caso "iguais" ? strings: "AAAA" e "AAAA"
comprimento lcs: 4 | uma subsequência: "AAAA"
---
caso "disjunto" ? strings: "ABCD" e "EFGH"
comprimento lcs: 0 | uma subsequência: ""
---
caso "vazio" ? strings: "" e "ABC"
comprimento lcs: 0 | uma subsequência: ""
---
caso "aggtab" ? strings: "AGGTAB" e "GXTXAYB"
comprimento lcs: 4 | uma subsequência: "GTAB"
---
caso "prefixo" ? strings: "abcdefghij" e "acegikmoq"
comprimento lcs: 5 | uma subsequência: "acegi"
---
caso "tema" ? strings: "programacao" e "dinamica"
comprimento lcs: 4 | uma subsequência: "amca"
---

comparativo de desempenho (tempo em ms)
-----
| caso | |s1| |s2| | lcs | | t_rec(ms) | | ops_rec | | t_dp(ms) | | ops_dp | | t_memo(ms) | | ops_memo |
-----
| classico | 7 | 6 | 4 | 0.034000 | 152 | 1.464700 | 42 | 0.041700 | 42 |
| iguais | 4 | 4 | 4 | 0.001900 | 5 | 0.005200 | 16 | 0.012300 | 5 |
| disjunto | 4 | 4 | 0 | 0.024400 | 139 | 0.006700 | 16 | 0.009200 | 33 |
| vazio | 0 | 3 | 0 | 0.000600 | 1 | 0.002300 | 0 | 0.001200 | 1 |
| aggtab | 6 | 7 | 4 | 0.074900 | 346 | 0.011700 | 42 | 0.013500 | 45 |
| prefixo | 10 | 9 | 5 | 3.123500 | 83726 | 0.018400 | 90 | 0.038000 | 176 |
| tema | 11 | 8 | 4 | 0.951900 | 15714 | 0.021500 | 88 | 0.035300 | 135 |
-----
PS C:\Users\joxto\Downloads\T1-ProjetoOtimizacaoAlgoritmos>

```

Execucao com casos padrao

Execucao com entrada pela linha de comando:

```

PS C:\Users\joxto\Downloads\T1-ProjetoOtimizacaoAlgoritmos> .\mvnw.cmd -q compile exec:java "-Dexec.args=ABCBDB BDCABA"
=====
lcs ? recursivo | dp (bottom-up) | memoização
=====
limiar recursivo: |s1|+|s2| <= 25 (acima: n/a)

caso "cli" ? strings: "ABCBDB" e "BDCABA"
comprimento lcs: 4 | uma subsequência: "BCBA"
---

comparativo de desempenho (tempo em ms)
-----
| caso | |s1| |s2| | lcs | | t_rec(ms) | | ops_rec | | t_dp(ms) | | ops_dp | | t_memo(ms) | | ops_memo |
-----
| cli | 7 | 6 | 4 | 0.030300 | 152 | 0.860200 | 42 | 0.038000 | 42 |
-----
PS C:\Users\joxto\Downloads\T1-ProjetoOtimizacaoAlgoritmos>

```

Execucao com caso informado pelo usuario

Exemplo de comando para executar os casos padrao:

```
./mvnw -q compile exec:java
```

Exemplo de comando para executar uma entrada especifica:

```
./mvnw -q compile exec:java "-Dexec.args=ABCBDB BDCABA"
```

Exemplo de comando para gerar o CSV do benchmark:

```
./mvnw -q compile exec:java "-Dexec.args=--csv"
```

Em ambiente Windows, os mesmos comandos podem ser executados com mvnw.cmd.

9. Resultados experimentais

Os tempos foram medidos com `System.nanoTime()` e apresentados em milissegundos. As operações foram contabilizadas com a classe `OperationCounter`.

Caso	Len s1	Len s2	LCS	Tempo rec. (ms)	Ops rec.	Tempo DP (ms)	Ops DP	Tempo memo (ms)	Ops memo
classico	7	6	4	0.010042	152	0.452250	42	0.016959	42
iguais	4	4	4	0.000542	5	0.001917	16	0.006791	5
disjunto	4	4	0	0.008583	139	0.001750	16	0.002958	33
vazio	0	3	0	0.000125	1	0.000250	0	0.000291	1
aggtab	6	7	4	0.023584	346	0.003375	42	0.004708	45
prefixo	10	9	5	1.212375	83726	0.006292	90	0.012709	176
tema	11	8	4	0.137750	15714	0.006584	88	0.007459	135

Os resultados mostram que a versão recursiva pura cresce rapidamente em número de operações. No caso prefixo, com strings de tamanhos 10 e 9, a versão recursiva realizou 83726 operações, enquanto a versão DP processou apenas 90 células internas. Isso confirma, na prática, a diferença entre uma solução exponencial e uma solução $O(m * n)$.

A versão DP apresenta comportamento previsível: o número de operações corresponde ao produto dos tamanhos das strings. Por exemplo, no caso clássico, $7 * 6 = 42$, exatamente o valor registrado em `ops_dp`. No caso vazio, como uma das strings tem tamanho 0, não há células internas a processar e `ops_dp = 0`.

A memoização também reduz drasticamente o trabalho em comparação com a recursão pura. Em alguns casos, como strings iguais, ela executa poucas chamadas porque os caracteres coincidem sucessivamente e a recorrência segue quase sempre pela diagonal. Em outros casos, visita mais estados, mas ainda evita a explosão de chamadas repetidas da força bruta.

10. Análise comparativa

O algoritmo recursivo puro é útil para explicar o problema, pois traduz diretamente a definição da LCS. No entanto, ele não é adequado para entradas grandes. Sua árvore de chamadas pode crescer muito rapidamente, principalmente quando há muitos caracteres diferentes e a recorrência precisa explorar dois caminhos em várias posições.

O algoritmo com programação dinâmica bottom-up é a abordagem mais adequada como solução principal do trabalho. Ele calcula cada subproblema uma vez, tem tempo polinomial e permite reconstruir uma subsequência comum máxima a partir da tabela preenchida.

A memoização fica entre as duas ideias: preserva a estrutura recursiva, mas armazena resultados já calculados. Na prática, ela também alcança complexidade $O(m * n)$, embora mantenha o custo da pilha recursiva e possa ter comportamento diferente na contagem de chamadas.

11. Testes e validação

Foram criados testes automatizados com JUnit. Os testes verificam:

- casos com strings vazias;
- strings iguais;
- strings sem caracteres em comum;
- caso clássico ABCBDAB e BDCABA;
- casos com um caractere comum;
- concordância entre recursivo, DP e memoização;
- reconstrução de uma LCS com comprimento correto;
- comportamento em escala para DP e memoização com strings de tamanho 500.

Esses testes ajudam a garantir que os três métodos retornam o mesmo comprimento de LCS nos casos em que todos são executados. Para entradas maiores, a validação concentra-se em DP e memoização, porque a recursão pura pode ser muito lenta.

12. Conclusão

O trabalho demonstrou que o problema da Maior Subsequência Comum se encaixa naturalmente em programação dinâmica. A solução recursiva pura é simples de entender, mas apresenta repetição intensa de subproblemas. A solução bottom-up elimina essa repetição usando uma matriz de estados e passa a ter complexidade $O(m * n)$, tornando a abordagem viável para entradas maiores.

Os resultados experimentais confirmam a análise teórica. Mesmo em casos pequenos, a diferença entre a quantidade de operações da recursão pura e da programação dinâmica já se torna evidente. A memorização também melhora o desempenho em relação à força bruta, mas a implementação bottom-up é especialmente adequada quando se deseja, além do tamanho, reconstruir uma subsequência comum máxima.